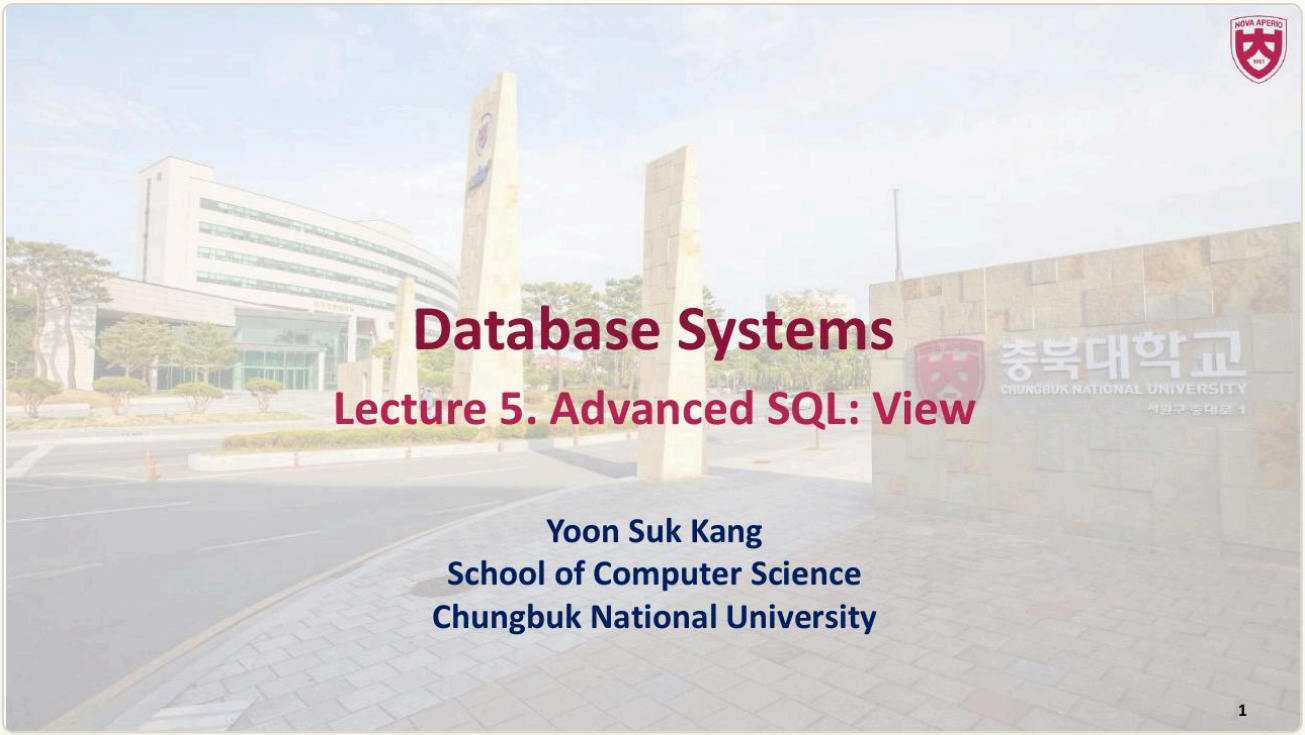


데이터베이스 시스템 — 5강: 고급 SQL — 뷰

- Yoon Suk Kang · 충북대 소프트웨어학부

✍ 메모

💡 이해 22쪽짜리 짧은 강의 — 뷰 정의 · 구현 2방식 · 갱신 가능 조건 · 권한

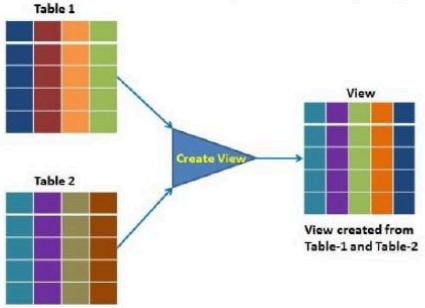
Concept of a View in SQL

❑ **What is a View in SQL?**

- A ***Virtual*** table ***derived*** from base tables
  - Not necessarily exist in a physical form

❑ **Operations on a View**

- Retrieval operations on a view does not have any limitations
- Update operations on a view are usually limited (i.e., a view is 'read-only')



2

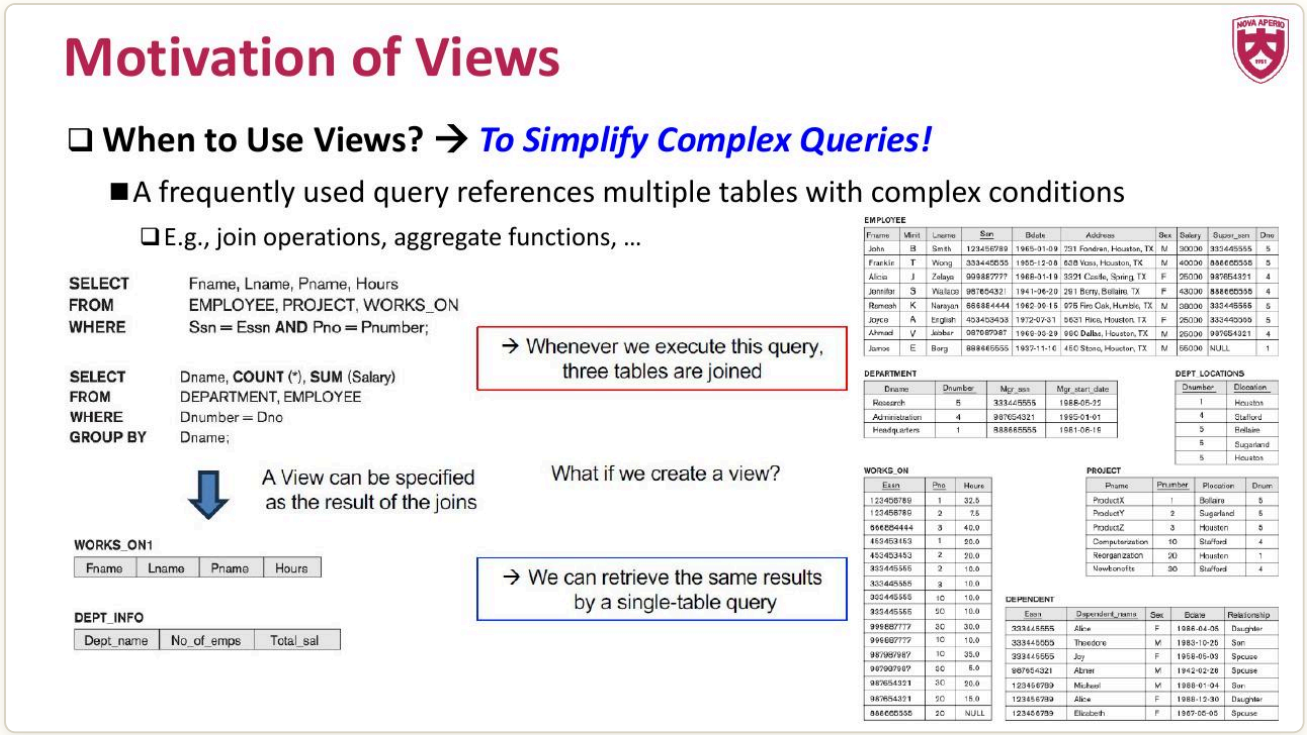
SQL의 뷰란?

- **기본 테이블(base table)에서 유도된 가상 테이블**
  - 물리적으로 존재하지 **않을 수도** 있다
- 뷰에 대한 연산
  - **검색**은 제한 없음
  - **갱신**은 보통 제한됨 — 뷰는 사실상 '읽기 전용'

✍ 메모

★ 시험 View = virtual table derived from base tables

⚠ 주의 뷰 = 복사본이 아니다 (2강 p.9와 연결)

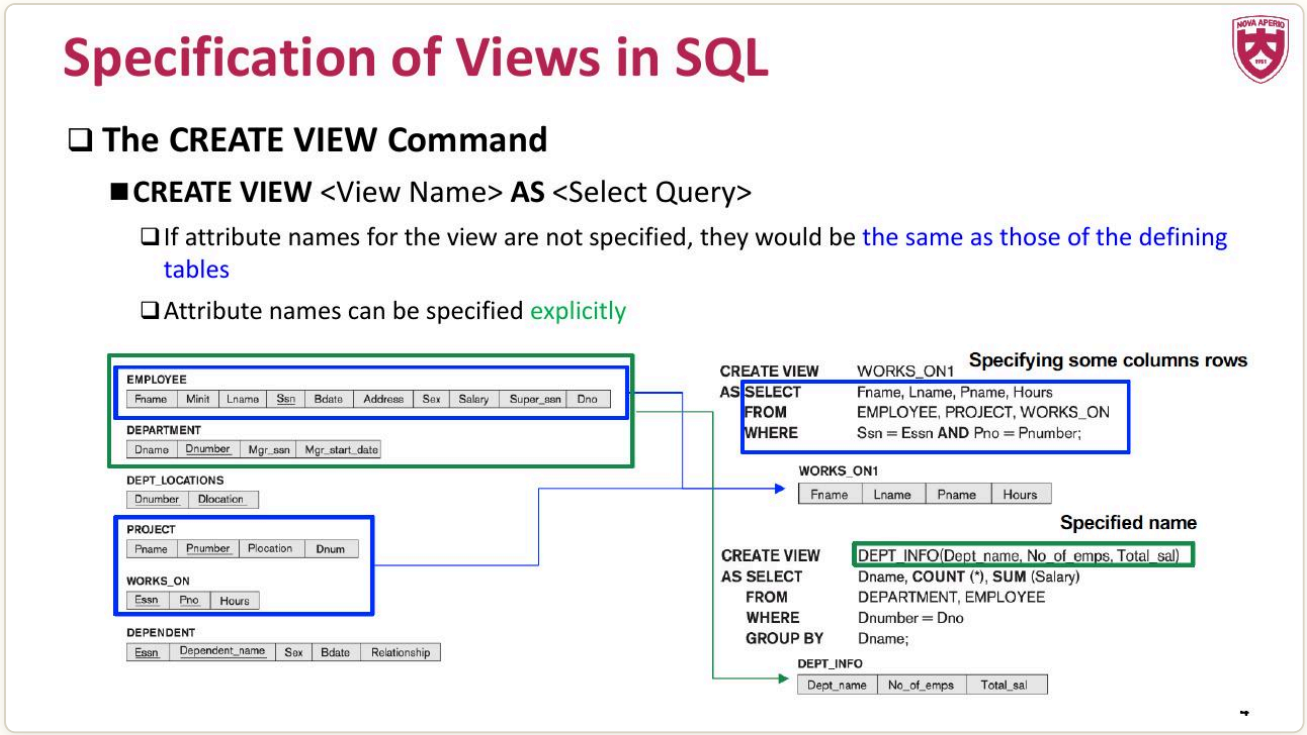


뷰를 쓰는 이유

- 언제 쓰나? → **복잡한 질의를 단순하게!**
  - 자주 쓰는 질의가 여러 테이블을 복잡한 조건으로 참조할 때 (조인, 집계 …)
  - 매번 세 테이블을 조인 → 조인 결과를 뷰로 두면 **한 테이블 질의**처럼

✍ 메모

— 넘어가도 되는 쪽



CREATE VIEW

```
CREATE VIEW <View Name> AS <Select Query>;
```

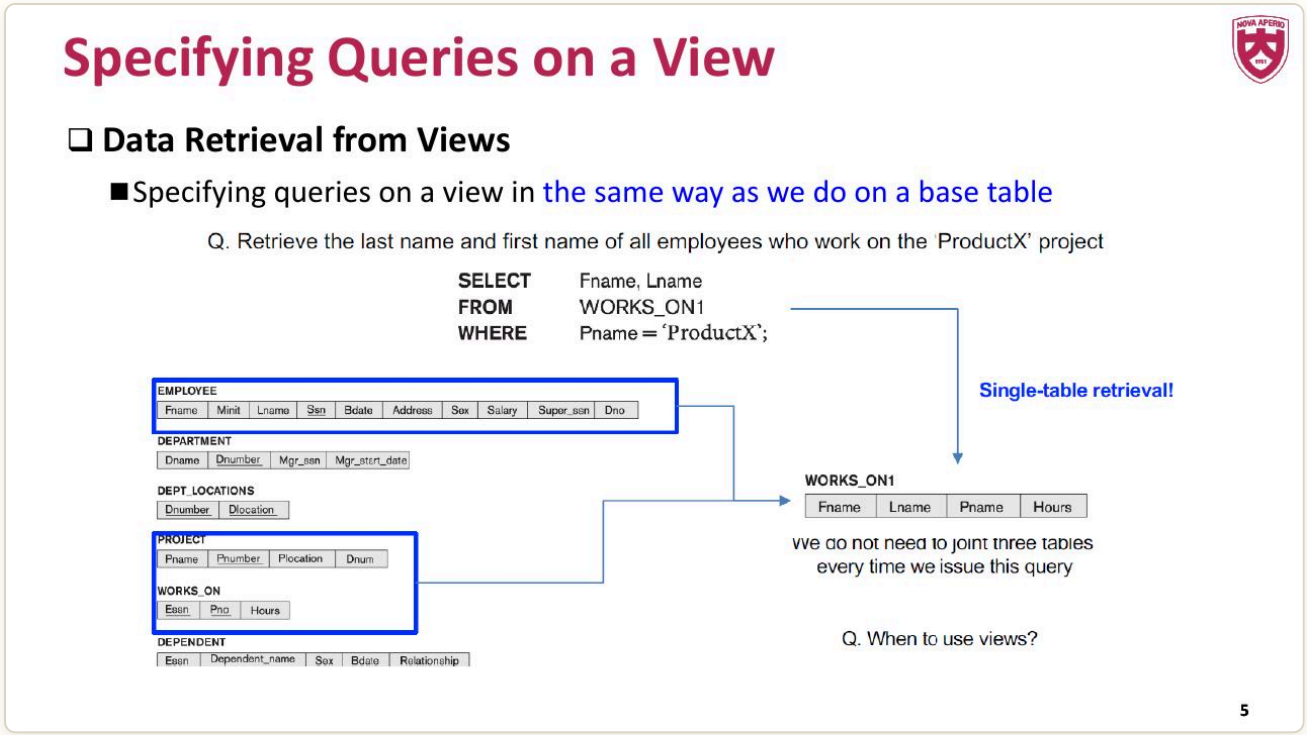
- 뷰의 속성 이름을 안 주면 → 정의 테이블의 속성 이름 그대로
- 속성 이름을 명시할 수도 있다

```
CREATE VIEW WORKS_ON1 AS SELECT Fname, Lname, Pname, Hours FROM EMPLOYEE, PROJECT, WORKS_ON
WHERE Ssn = Essn AND Pno = Pnumber;
```

```
CREATE VIEW DEPT_INFO(Dept_name, No_of_emps, Total_sal) AS SELECT Dname, COUNT(*),
SUM(Salary) FROM DEPARTMENT, EMPLOYEE WHERE Dnumber = Dno GROUP BY Dname;
```

✍ 메모

★ 시험 WORKS_ON1(조인 뷰)와 DEPT_INFO(집계 뷰) — 뒤의 '갱신 불가' 예로 다시 나옴



뷰에 대한 질의

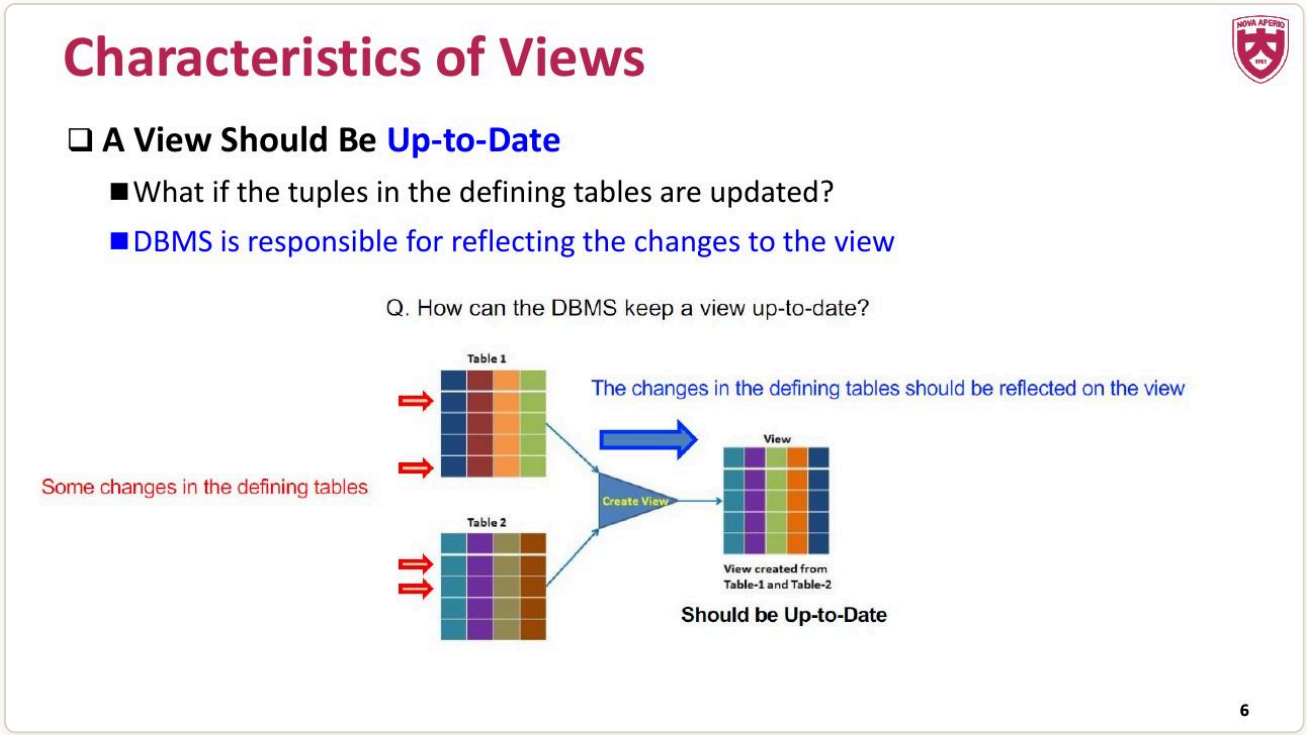
- 기본 테이블과 **같은 방식**으로 질의

```
SELECT Fname, Lname FROM WORKS_ON1 WHERE Pname = 'ProductX';
```

  - → 매번 세 테이블을 조인할 필요가 없다

✍ 메모

— 넘어가도 되는 쪽



뷰의 특성

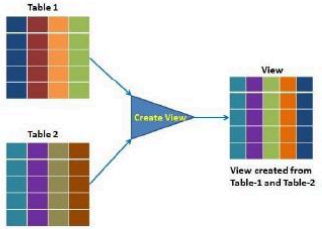
- 뷰는 **최신 상태(up-to-date)** 여야 한다
  - 정의 테이블의 튜플이 바뀌면?
  - → 변경을 뷰에 반영하는 것은 **DBMS의 책임**

✍ 메모

★ 시험 최신 유지 = DBMS 책임 (사용자가 아님)

View Implementation in the DBMS

❑ **Two Approaches**
- ■ Query modification
- ■ View materialization



❑ **Query Modification (Approach 1)**
- ■ Transforming the view query into a query on the underlying base tables
  - ❑ Not maintaining views physically

```
CREATE VIEW   WORKS_ON1
AS SELECT     Fname, Lname, Pname, Hours
  FROM        EMPLOYEE, PROJECT, WORKS_ON
  WHERE       Ssn = Essn AND Pno = Pnumber;

SELECT  Fname, Lname
FROM    WORKS_ON1
WHERE   Pname = 'ProductX';
```

Transforming the view query

```
SELECT  Fname, Lname
FROM    EMPLOYEE, PROJECT, WORKS_ON
WHERE   Ssn = Essn AND Pno = Pnumber
        AND Pname = 'ProductX';
```

→ Not aligned with the philosophy of Views in SQL

Can be convenient to users,
But not to the DB system

7

DBMS의 뷰 구현 ① Query Modification

- 두 가지 방법: **Query modification** / **View materialization**
- **Query modification(질의 변경)**
  - 뷰 질의를 **기본 테이블에 대한 질의로 바꿔서** 실행
  - 뷰를 물리적으로 **유지하지 않는다**

```
SELECT Fname, Lname FROM WORKS_ON1 WHERE Pname = 'ProductX';
```

```
→ SELECT Fname, Lname FROM EMPLOYEE, PROJECT, WORKS_ON WHERE Ssn = Essn AND Pno = Pnumber
AND Pname = 'ProductX';
```

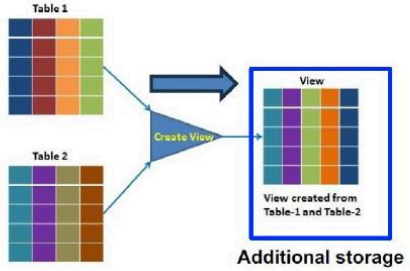
✍ 메모

💡 **이해** 사용자는 편하지만, 복잡한 뷰를 자주 쓰면 매번 조인 → 비효율

View Implementation in the DBMS (cont.)

❑ **View Materialization (Approach 2)**

- Physically create a view table when the view is first queried
- Keep that a virtual table "for a while" (how long?)
  - ❑ **Assumption**: *other retrieval queries on the view will follow frequently (hopefully)*
- **Incremental updates**
  - ❑ When an update is applied to ***one of the defining tables***, it should be reflected to the view *incrementally*



Q. When the defining tables are updated, How/when to update the view?

8

DBMS의 뷰 구현 ② View Materialization

- 뷰를 처음 질의할 때 **뷰 테이블을 물리적으로 만든다**
  - 그 가상 테이블을 '한동안' 유지 (얼마나?)
    - 가정: 그 뷰에 대한 검색이 (바라건대) 자주 이어진다
  - **Incremental update(점진적 갱신)**
    - 정의 테이블이 갱신되면 뷰에 **조금씩** 반영

✍ 메모

★ 시험 두 방식 대비: 변환해서 매번 계산 vs 만들어 두고 점진 갱신

View Implementation in the DBMS (cont.)

❑ When/How to Update the View?

■ Immediate update

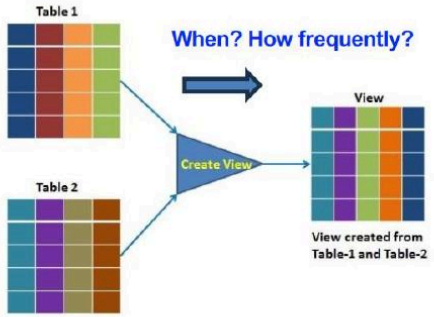
- ❑ Update a view *as soon as the defining base tables are updated*
- ❑ Always up-to-date, but sometime unnecessary and inefficient

■ Lazy update

- ❑ Update a view when it is needed by a view query
- ❑ Not up-to-date until a query is specified on the view

■ Periodic update

- ❑ Updates a view *periodically*
- ❑ A view query may get a result that is not up-to-date



9

뷰를 언제·어떻게 갱신할까?

- **Immediate update** — 기본 테이블이 바뀌는 즉시 뷰 갱신
  - 늘 최신이지만 때로 불필요하고 비효율적
- **Lazy update** — 뷰 질의가 필요할 때 갱신
  - 질의하기 전까지는 최신이 아님
- **Periodic update** — 주기적으로 갱신
  - 뷰 질의 결과가 최신이 아닐 수 있음

✍ 메모

★ 시험 갱신 시점 3가지: Immediate · Lazy · Periodic — 장단점 한 줄씩

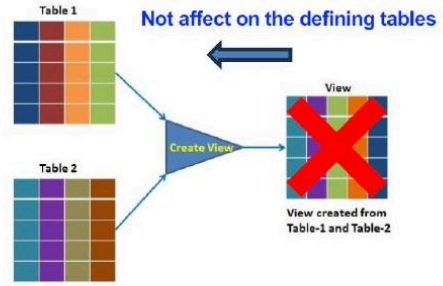
Dropping a View in SQL

❑ The DROP VIEW Command

■ We can use the **DROP VIEW** command if we do not need a view anymore

❑ The result of this command does not affect on its defining tables

DROP VIEW WORKS_ON1;



Q. What if the view is not queried for a certain period of time?
→ Remove the view

Q. What if the view is needed in the future?
→ Recompute it from scratch

10

DROP VIEW

- 더 필요 없으면 **DROP VIEW**

```
DROP VIEW WORKS_ON1;
```

  - **정의 테이블에는 영향 없음**

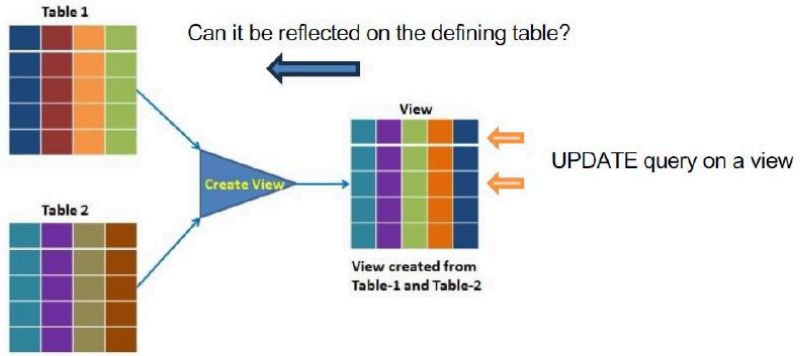
✍ 메모

⚠ 주의 **뷰를 지워도 원래 표의 데이터는 그대로**

Update Operations on Views

❑ Can a View be Updated?

- We can always issue any ***retrieval queries*** against any views
- INSERT, DELETE, or UPDATE command on a view is not possible in many cases



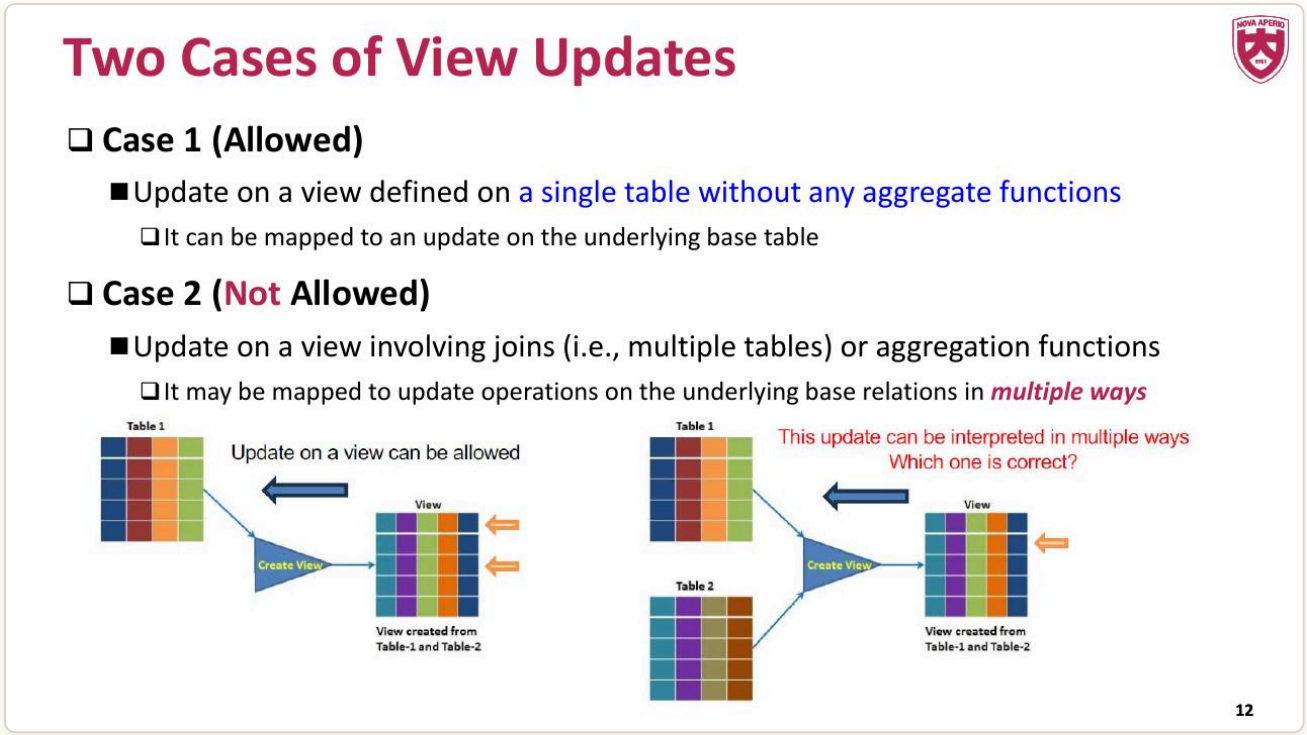
11

뷰에 대한 갱신 연산

- 뷰가 갱신될 수 있나?
  - **검색** 질의는 어떤 뷰에든 언제나 가능
  - INSERT·DELETE·UPDATE는 **많은 경우 불가능**

✍ 메모

— 넘어가도 되는 쪽

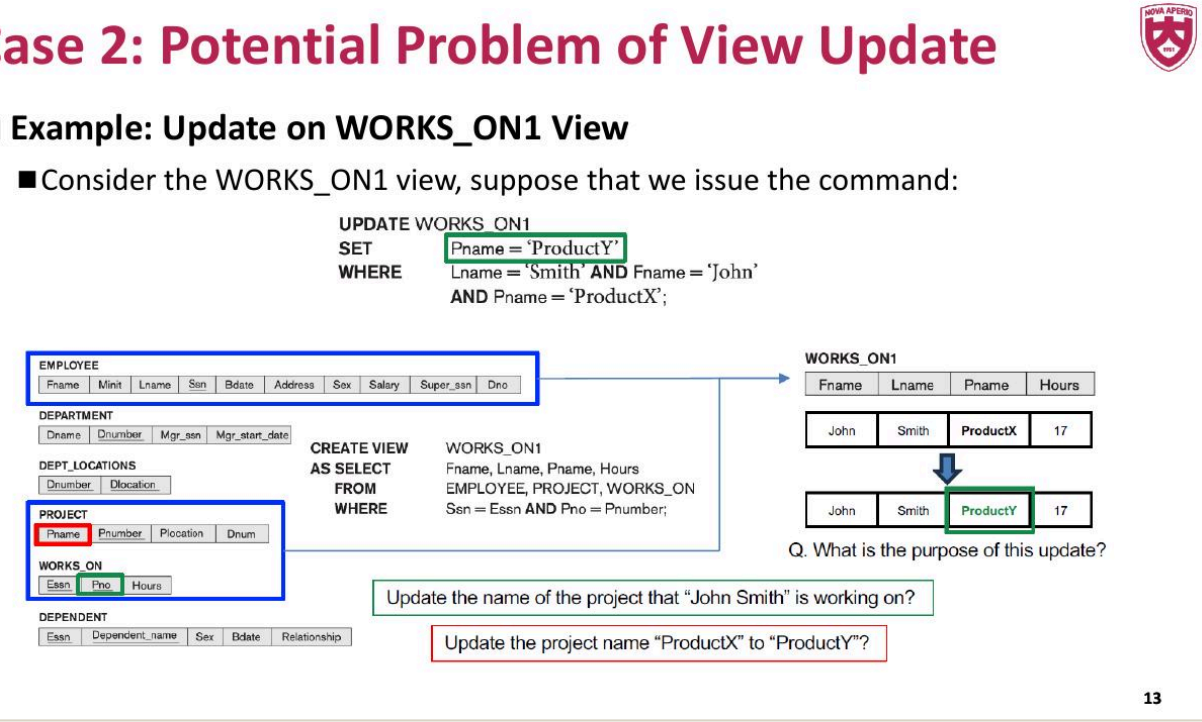


뷰 갱신의 두 경우

- **Case 1 (허용)**: 집계 함수 없는 **단일 테이블** 뷰의 갱신
  - 기본 테이블의 갱신으로 매핑할 수 있다
- **Case 2 (불허)**: **조인(여러 테이블)이나 집계**를 쓴 뷰의 갱신
  - 기본 릴레이션의 갱신으로 **여러 방식**으로 매핑될 수 있다

✍ 메모

★ 시험 단일 표 + 집계 없음 → 갱신 가능 / 조인·집계 → 불가



Case 2의 문제 — WORKS_ON1 갱신

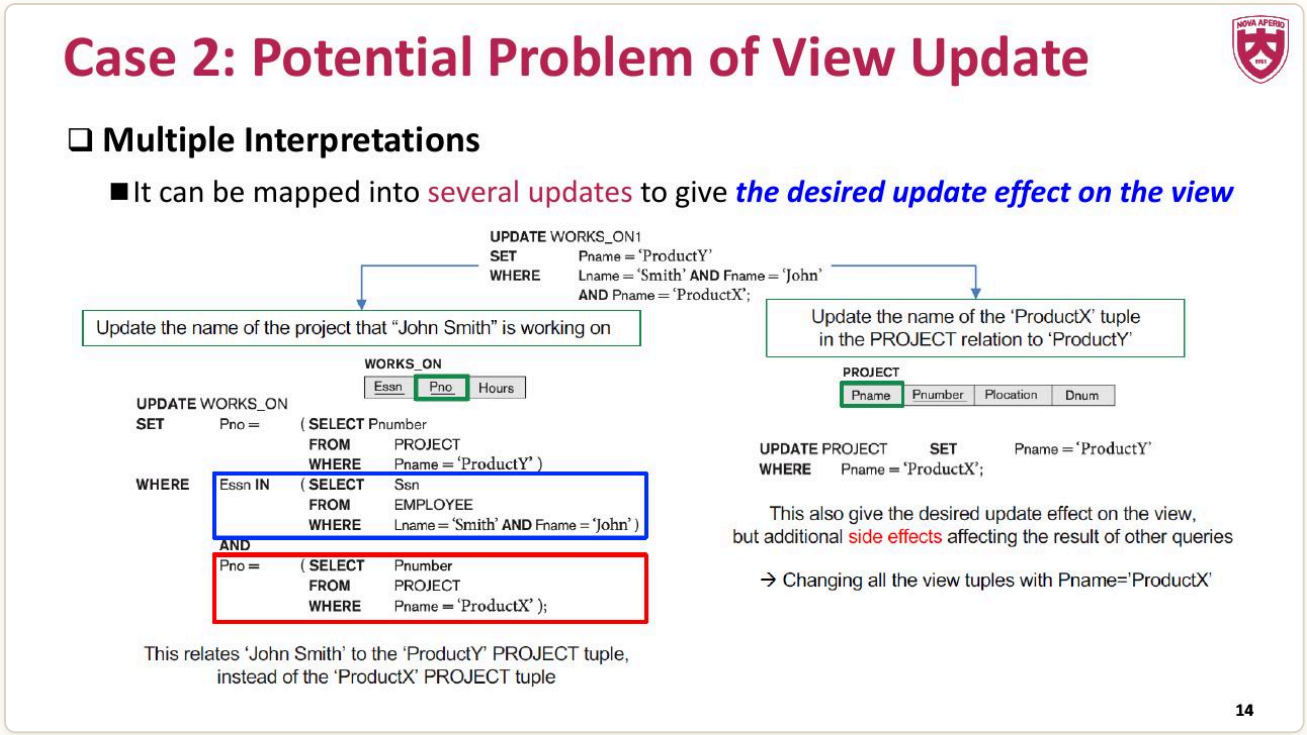
- WORKS_ON1 뷰에 다음 명령을 내리면?

```
UPDATE WORKS_ON1 SET Pname = 'ProductY' WHERE Lname = 'Smith' AND Fname = 'John' AND Pname
= 'ProductX';
```

  - 목적이 뭘까?
  - 'John Smith가 일하는 프로젝트를 바꾸기'? 아니면 '프로젝트 이름 ProductX를 ProductY로 바꾸기'?

✍ 메모

— 넘어가도 되는 쪽



Case 2의 문제 — 여러 해석

- 뷰에 원하는 효과를 내는 기본 테이블 갱신이 **여러 개**
- **해석 1**: WORKS_ON에서 Smith의 프로젝트 번호를 ProductY로

```
UPDATE WORKS_ON SET Pno = (SELECT Pnumber FROM PROJECT WHERE Pname = 'ProductY') WHERE Essn
IN (SELECT Ssn FROM EMPLOYEE WHERE Lname = 'Smith' AND Fname = 'John') AND Pno = (SELECT
Pnumber FROM PROJECT WHERE Pname = 'ProductX');
```

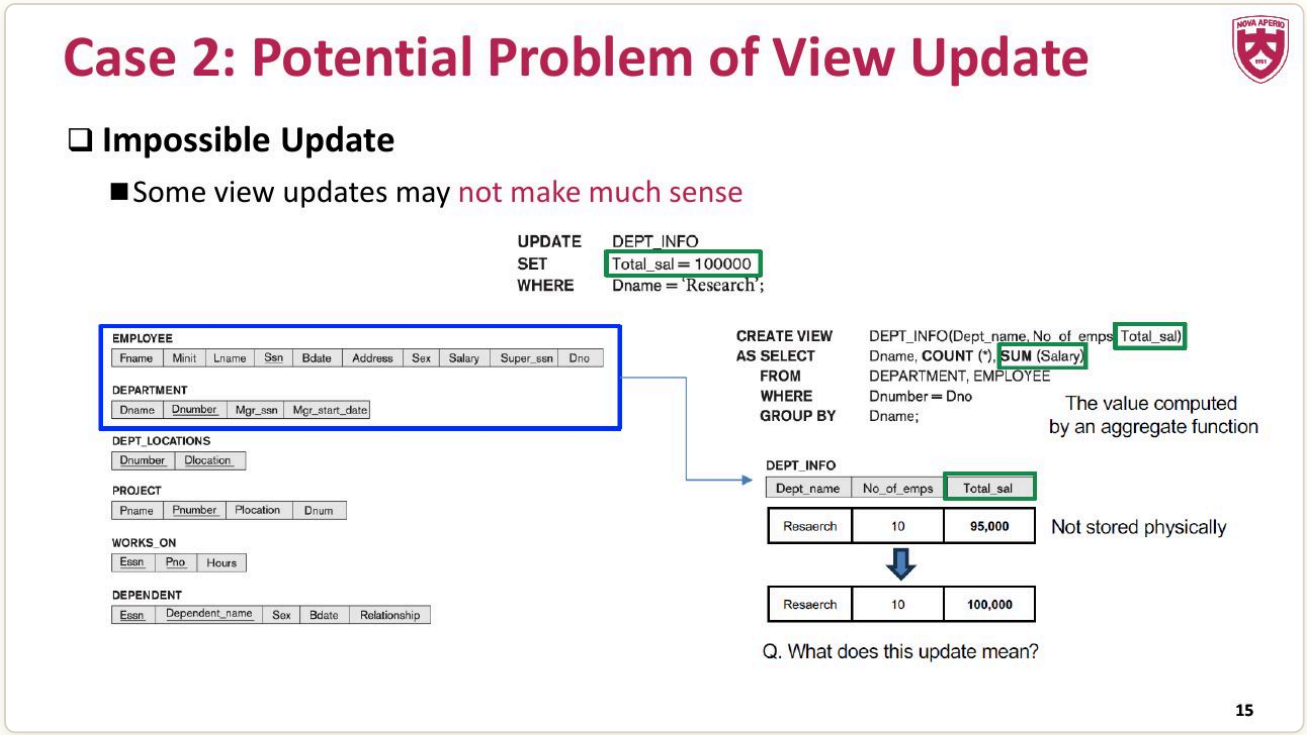
- **해석 2**: PROJECT의 이름 자체를 바꿈

```
UPDATE PROJECT SET Pname = 'ProductY' WHERE Pname = 'ProductX';
```

  - 원하는 효과는 나지만 **부작용** — ProductX였던 모든 뷰 튜플이 바뀜

✍ 메모

✏ 예·풀이 해석 2는 Smith 말고 ProductX에서 일하던 다른 직원의 행까지 바뀜



Case 2의 문제 — 불가능한 갱신

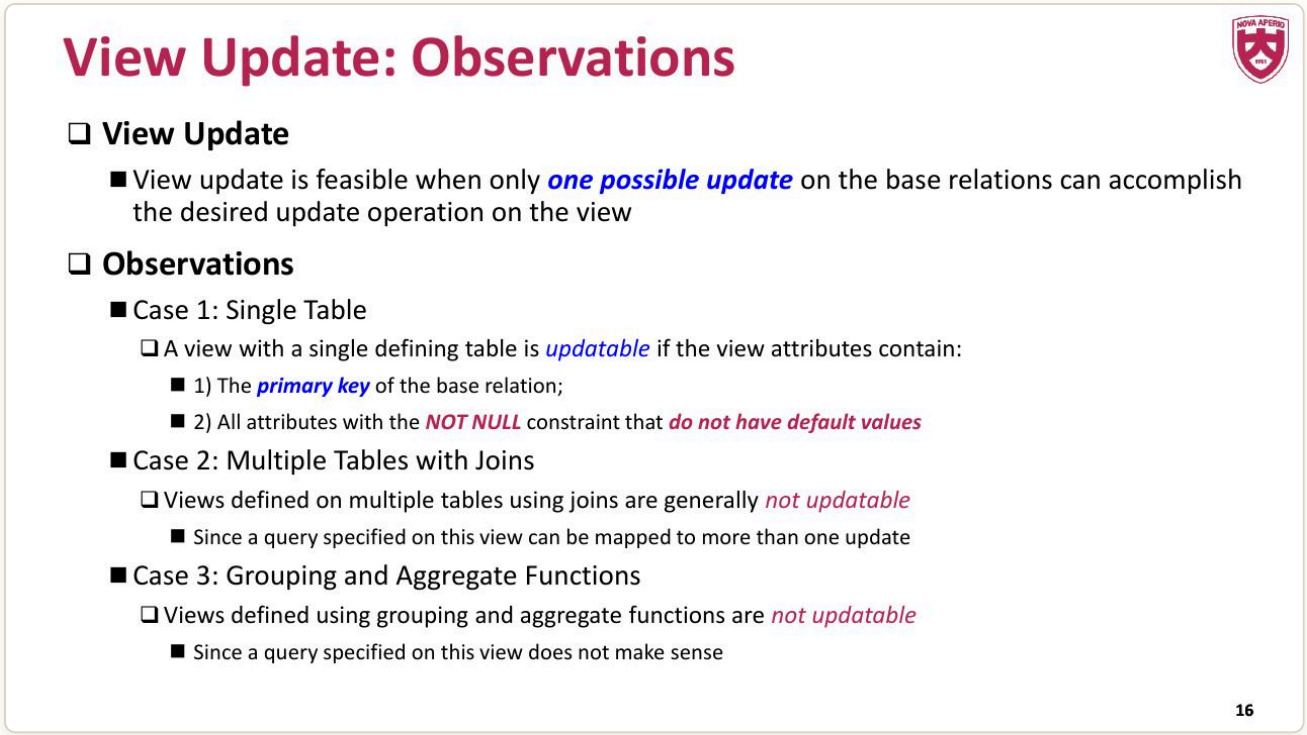
- 말이 안 되는 뷰 갱신도 있다

```
UPDATE DEPT_INFO SET Total_sal = 100000 WHERE Dname = 'Research';
```

  - Total_sal은 **집계로 계산된 값** (물리적으로 저장되지 않음)
  - 95,000 → 100,000으로 바꾸라는 게 무슨 뜻? — 누구 월급을 얼마나?

✍️ 메모

★ 시험 **집계 뷰 갱신 = 의미 없음 (impossible update)**



뷰 갱신 — 관찰

- 뷰 갱신이 가능한 때: 원하는 효과를 내는 기본 릴레이션 갱신이 **단 하나**일 때
- **Case 1 단일 테이블**: 뷰 속성에 다음이 있으면 갱신 가능
  - 1) 기본 릴레이션의 **기본키**
  - 2) **기본값 없는 NOT NULL 속성 전부**
- **Case 2 조인을 쓴 여러 테이블**: 일반적으로 **갱신 불가** — 둘 이상의 갱신으로 매핑될 수 있어서
- **Case 3 그룹·집계**: **갱신 불가** — 말이 안 돼서

✍ 메모

★ 시험 단일 테이블 뷰 갱신 조건 2개(PK + NOT NULL·무기본값 속성) — 서술형 단골

💡 이해 이유: 뷰에 INSERT하면 빠진 속성은 NULL/기본값으로 채워야 하는데, PK·NOT NULL이 비면 안 되니까

Views as Authorization

❑ Authorization Mechanisms using Views

- ■ Views can be used to hide **certain attributes** or **tuples** from unauthorized users.
- ■ Example 1: DEPT5EMP
  - ❑ The manager of department 5 is only allowed to see information for employees working for department 5
    - ■ Create the view 'DEPT5EMP' and grant the user the privilege to query the view but not EMPLOYEE
- ■ Example 2: BASIC_EMP_DATA
  - ❑ Similarly, a view can restrict a user to only see certain columns
  - ❑ Normal employees can see only the basic data: first name, last name, and address of an employee

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

```
CREATE VIEW   DEPT5EMP   AS
SELECT        *
FROM          EMPLOYEE
WHERE         Dno = 5;

CREATE VIEW   BASIC_EMP_DATA   AS
SELECT        Fname, Lname, Address
FROM          EMPLOYEE;
```

17

권한 부여로서의 뷰

- 권한 없는 사용자에게서 특정 **속성(열)이나 튜플(행)을 숨긴다**
- **Example 1: DEPT5EMP** — 행 숨기기
  - 5번 부서 관리자는 5번 부서 직원 정보만 보게
  - 뷰를 만들고, 그 뷰에 대한 질의 권한만 주고 EMPLOYEE 권한은 안 준다

```
CREATE VIEW DEPT5EMP AS SELECT * FROM EMPLOYEE WHERE Dno = 5;
```

- **Example 2: BASIC_EMP_DATA** — 열 숨기기
  - 일반 직원은 이름·성·주소만

```
CREATE VIEW BASIC_EMP_DATA AS SELECT Fname, Lname, Address FROM EMPLOYEE;
```

✍ 메모

★ 시험 WHERE로 행 제한, SELECT 목록으로 열 제한

Exercise: Views in SQL

❑ Exercise 1

- ■ Specify the following view in SQL on the COMPANY database schema
- ■ Explain when the view can be useful (i.e., guess the purpose)

A view that has the department name, manager name, and manager salary for every department

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

18

연습 1: 뷰 만들기

- COMPANY 스키마에 다음 뷰를 SQL로 정의하고
- 그 뷰가 언제 쓸모 있을지 설명하라 (목적 추측)
  - **모든 부서**의 부서 이름, 관리자 이름, 관리자 월급

✍ 메모

✏ 예·풀이 정답 p.21 — DEPARTMENT와 EMPLOYEE를 Mgr_ssn = Ssn으로 조인

Exercise: Views in SQL

❑ Exercise 2

- ■ Specify the following view in SQL on the COMPANY database schema
- ■ Explain when the view can be useful (i.e., guess the purpose)

A view that has the project name, controlling department name, number of employees, and total hours worked per week on the project for each project

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

DEPT_LOCATIONS

Dnumber	Dlocation

PROJECT

Pname	Pnumber	Plocation	Dnum

WORKS_ON

Essn	Pno	Hours

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship

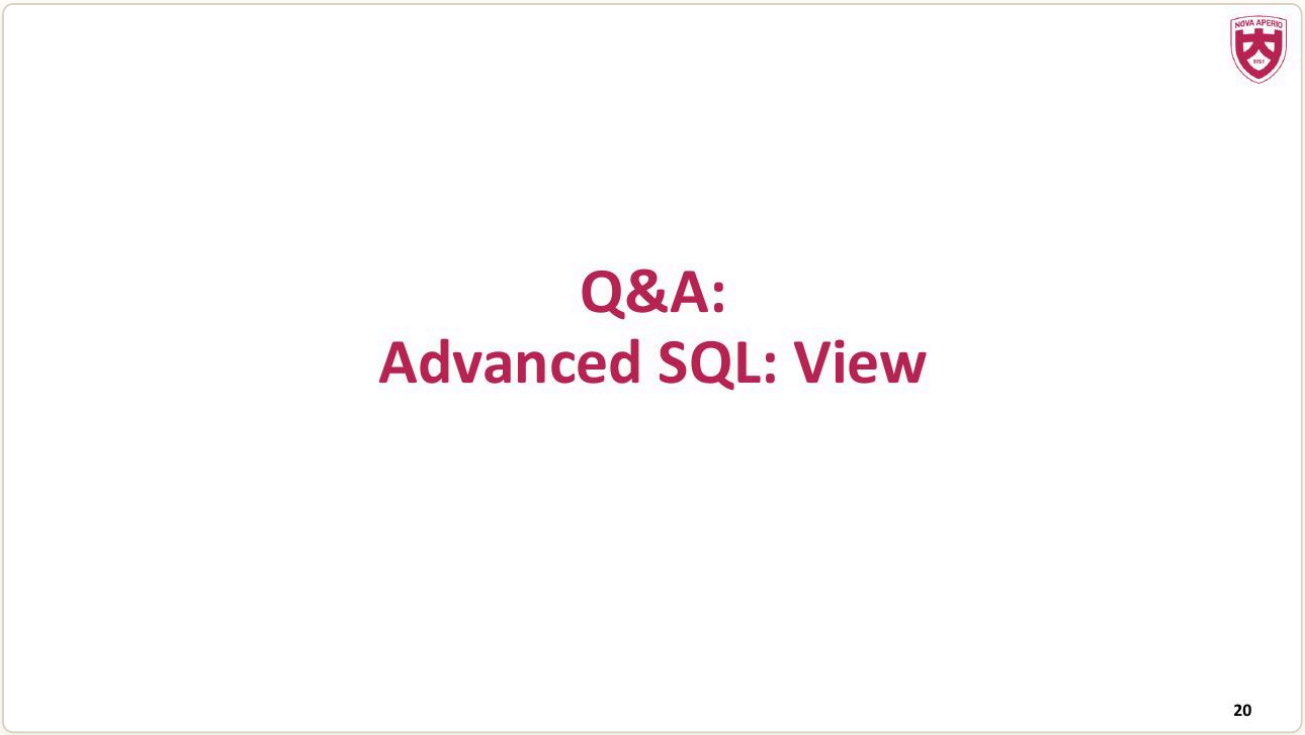
19

연습 2: 뷰 만들기

- 각 프로젝트의 이름, 통제 부서 이름, 일하는 직원 수, 주당 총 근무 시간

✍ 메모

✏ 예·풀이 정답 p.22 — 3개 표 조인 + GROUP BY + COUNT·SUM

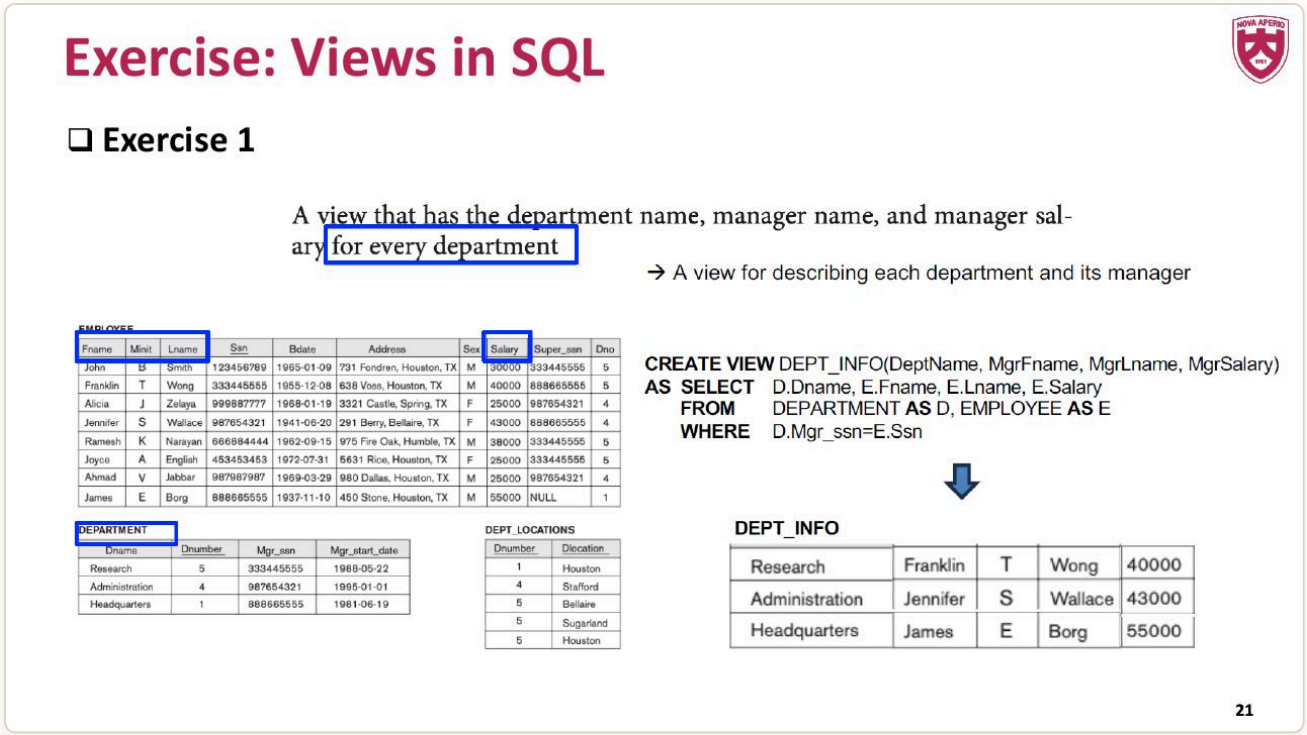


Q&A

- Advanced SQL: View — 끝
- p.21~22는 연습문제 정답

✍ 메모

— 넘어가도 되는 쪽



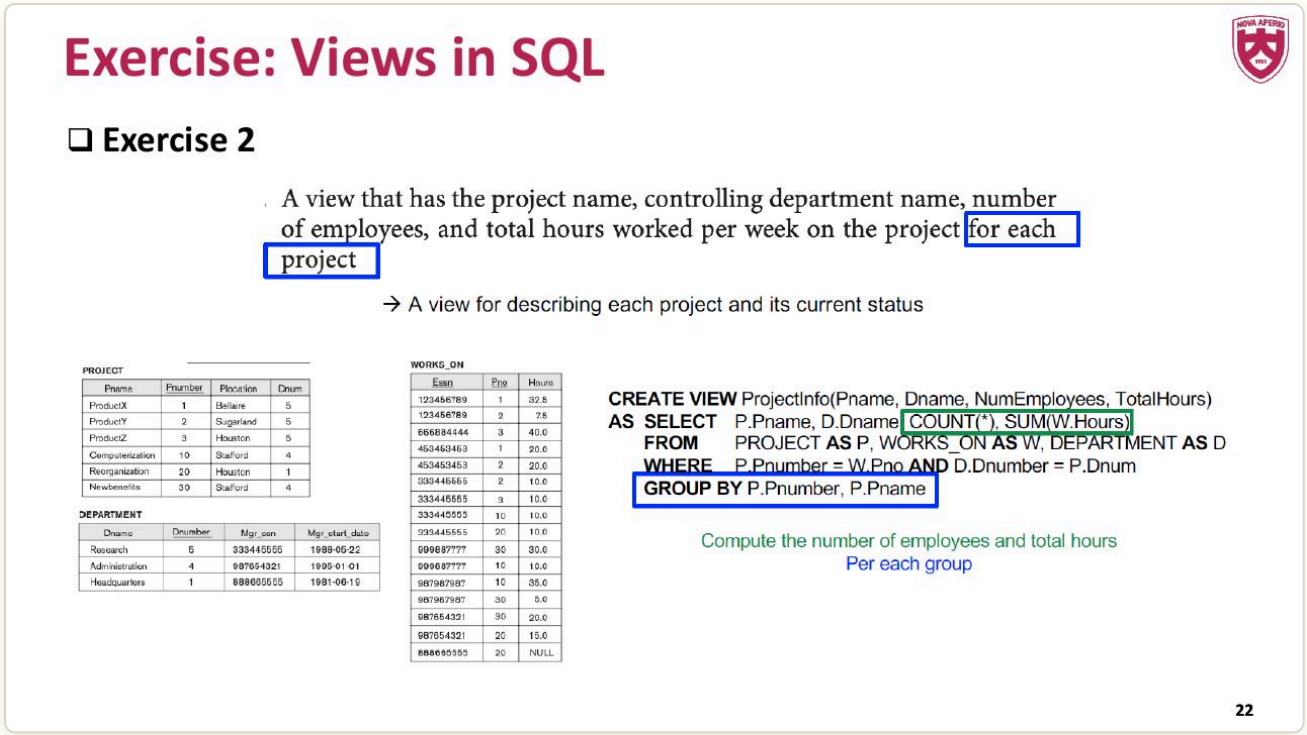
연습 1 정답

```
CREATE VIEW DEPT_INFO(DeptName, MgrFname, MgrLname, MgrSalary) AS SELECT D.Dname, E.Fname, E.Lname, E.Salary FROM DEPARTMENT AS D, EMPLOYEE AS E WHERE D.Mgr_ssn = E.Ssn;
```

- 결과: (Research, Franklin Wong, 40000), (Administration, Jennifer Wallace, 43000), (Headquarters, James Borg, 55000)
- 목적: 각 부서와 그 관리자를 설명하는 뷰

✍️ 메모

⚠ 주의 조인 뷰 → 갱신 불가(Case 2)



연습 2 정답

```
CREATE VIEW ProjectInfo(Pname, Dname, NumEmployees, TotalHours) AS SELECT P.Pname, D.Dname, COUNT(*), SUM(W.Hours) FROM PROJECT AS P, WORKS_ON AS W, DEPARTMENT AS D WHERE P.Pnumber = W.Pno AND D.Dnumber = P.Dnum GROUP BY P.Pnumber, P.Pname;
```

- 목적: 각 프로젝트와 현재 상태를 설명하는 뷰 — 그룹마다 인원과 총 시간

✍ 메모

⚠ 주의 교안 GROUP BY에 D.Dname이 없다 — 표준 SQL에선 D.Dname도 넣어야 안전

★ 시험 집계 뷰 → 갱신 불가(Case 3)